

ورا كل تقييم فيه إحساس... تخيل تقدر تستفيد منه وتفهم الناس بذكاء؟💡
تحليل المشاعر (Sentiment Analysis) مو مجرد أداة تقنية، هو بوابة لفهم السوق، العملاء، وحتى المجتمع.
الثريد هذا بيشرح لك الفكرة بأسلوب بسيط وواضح... احفظه وارجع له وقت تركيزك👉⌚!



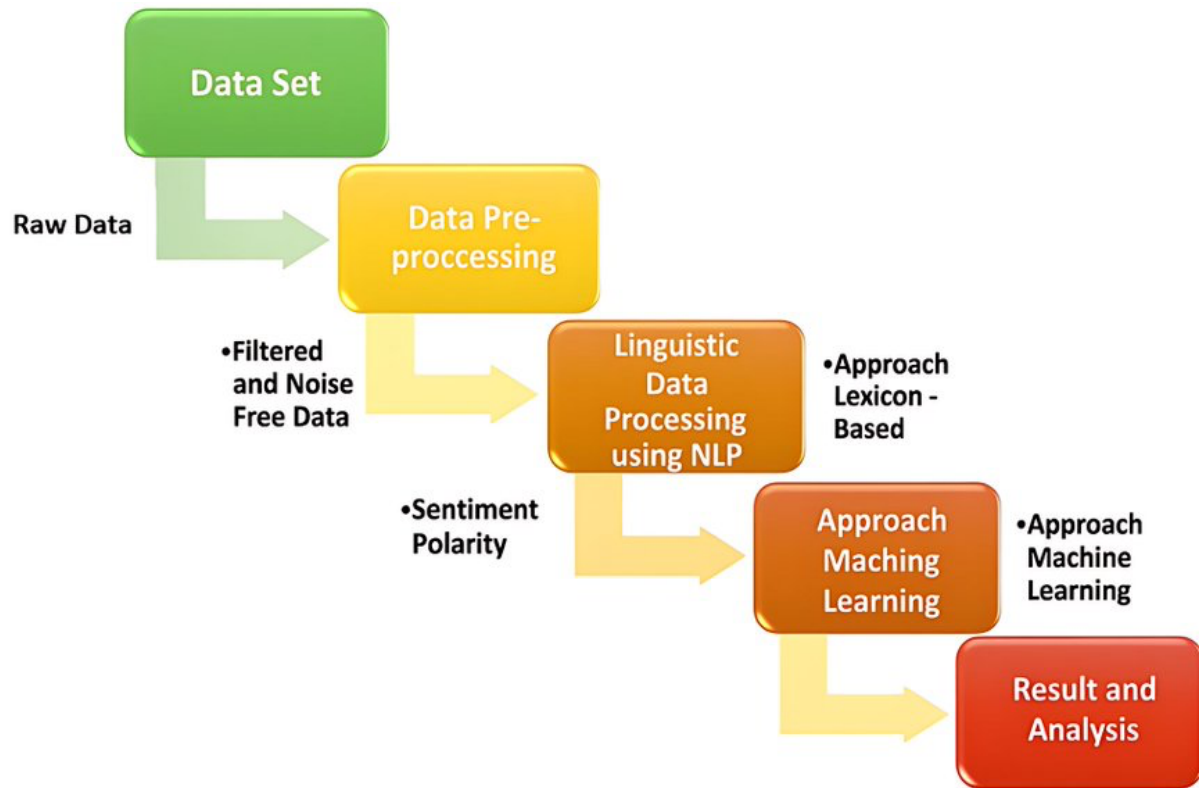
طبيب، وش هو تحليل المشاعر (Sentiment Analysis) بالضبط؟🤖 هو مجال فرعي من معالجة اللغات الطبيعية (NLP) هدفه الأساسي تحليل النصوص عشان نفهم مشاعر الكاتب أو موقفه تجاه موضوع معين يعني هل هو إيجابي، سلبي، محايد، أو حتى مشاعر أدق زي سعيد😊، حزين😞، غاضب😡، وغيره. بنتعلم كيف نبني موديل يسوي هالشي.



Sentiment Analysis

وليش تحليل المشاعر مهم؟ لأن آراء الناس اليوم في التقييمات، منصات التواصل الاجتماعي، وغيرها صارت مصدر معلومات ذهبي تقدر من خلالها تفهم جمهورك، تحسّن منتجك، وتتخذ قرارات أذكى. والتقنيات الحديثة سهّلت علينا نجمّع هالبيانات ونحلّلهما بسرعة وفعالية

- في الثريد بإذن الله بنغطي سوا عدة مواضيع أساسية عشان نقدر نبني نظام تحليل مشاعر ■ تنظيف وتجهيز البيانات النصية ✨
- بناء متجهات الخصائص (Feature Vectors) من النصوص ■ تدريب موديل تعلم آلة لتصنيف المراجعات (إيجابي/سلبي)
- التعامل مع بيانات نصية كبيرة ⚡ (Out-of-core learning) استخلاص المواضيع الرئيسية من مجموعة مستندات 😊 (Topic Modeling)



ولتطبيق تعلم الآلة على تحليل المشاعر، أول وأهم خطوة؟ أكيد البيانات 😎! ينطبق على مثال عملي ببيانات مشهورة في هالمجال تقييمات الأفلام من IMDb. بنشوف كيف نجيبها ونجهزها خطوة بخطوة عشان تكون الصورة واضحة تمامًا 😊.

البيانات اللي بنشتغل عليها هي مجموعة مراجعات الأفلام الكبيرة (Large Movie Review Dataset). هذي المجموعة فيها 50 ألف تقييم حقيقي، مقسمة لـ "إيجابي" و "سلبي"، وموجودة في مجلدات train/test ومجلدات فرعية positive/negative. نحمل البيانات هذي كملف `acllmbd_v1.tar.gz` من موقع ستانفورد. بعد التحميل، تقدر تفك الضغط بنفسك، أو تستخدم الـ Python ومكتبة الـ `tarfile`.

```

1 import tarfile
2 import os
3
4 filename = 'aclImdb_v1.tar.gz' # اسم الملف الذي حملناه
5 extract_path = '.' # المسار الذي بنفك فيه الملف (النقطة تعني المجلد الحالي)
6
7 # نتأكد إن الملف موجود أصلاً
8 if os.path.exists(filename):
9     try:
10         # نطبع رسالة للمستخدم
11         print(f'Extracting file: {filename}...')
12         # مضاووط بـ tar نفتح الملف للقراءة كملف
13         with tarfile.open(filename, 'r:gz') as tar:
14             # نفك كل محتوياته للمسار المحدد
15             tar.extractall(path=extract_path)
16         # نطبع رسالة نجاح
17         print('File extracted successfully! 🎉')
18     except Exception as e:
19         # نطبع أي خطأ ممكن يصير
20         print(f'Error during extraction: {e}')
21 else:
22     # نطبع خطأ لو الملف مو موجود
23     print(f'Error: File {filename} not found. Please download it first.')

```

بعد فك الضغط، هدفنا نجمع الـ 50 ألف ملف نصي في DataFrame واحد. هذا الكود يبدأ العملية بتعريف المسارات وتجهيز list لتجميع البيانات ثم يبدأ الـ Loops

```

Started assembling reviews...
Reading files from: aclImdb\test\pos...
Reading files from: aclImdb\test\neg...
Reading files from: aclImdb\train\pos...
Reading files from: aclImdb\train\neg...
Finished assembly! Total reviews: 50000.

```

```

1 import pandas as pd
2 import os
3
4 basepath = 'aclImdb' # المسار الرئيسي للمجلدات بعد ذلك المسار
5 labels = {'pos': 1, 'neg': 0} # لترجمة اسم المجلد (pos/neg) إلى رقم (1/0)
6 all_data = [] # قائمة لتجميع فيها البيانات list
7
8 print('Started assembling reviews...') # طباعة
9 # Loop على مجلدات train , test
10 for s in ('test', 'train'):
11     # Loop على مجلدات pos و neg
12     for l in ('pos', 'neg'):
13         # تركيب المسار الكامل للمجلد الحالي
14         path = os.path.join(basepath, s, l)
15         if not os.path.isdir(path):
16             print(f'Warning: Directory not found {path}') # طباعة
17             continue # لتخطي المسار غير موجود
18
19         print(f'Reading files from: {path}...') # طباعة
20         file_list = os.listdir(path) # قائمة بكل الملفات داخل المجلد
21
22         # Loop على كل ملف داخل المجلد الحالي
23         for file in file_list:
24             file_path = os.path.join(path, file) # المسار الكامل للملف
25             try:
26                 # يوم لتضمين utf-8 يفتح الملف للقراءة و يترجم
27                 with open(file_path, 'r', encoding='utf-8') as infile:
28                     txt = infile.read() # اقرأ الملف
29                     # list حقه تحت label يجمع النص والـ
30                     all_data.append([txt, labels[l]])
31             except Exception as e:
32                 # لم يفتح الملف أو خطأ في القراءة أو ملف مشغول
33                 print(f'Error reading file {file_path}: {e}') # طباعة
34
35 # DataFrame من list تحول إلى DataFrame بعد انتهاء كل الـ
36 df = pd.DataFrame(all_data, columns=['review', 'sentiment'])
37 print(f'Finished assembly! Total reviews: {len(df)}.') # طباعة

```

الحين الـ DataFrame (df) جاهز لكنه مرتب. نخلطه (Shuffle) عشان نضمن العشوائية، ونحفظه كـ CSV عشان نستخدمه بعدين

```

1 import numpy as np
2 # تعريف من الخطوة السابقة df و pd
3
4 # --- الخلط (Shuffle) ---
5 # np.random.seed(0) # (اختياري) يمكن استخدامه لضمان نفس الخلط كل مرة (للتجارب)
6 print('Shuffling data...') # طباعة
7 # DataFrame ونطابق على الـ indices لنظمي ترتيب عشوائي
8 df_shuffled = df.reindex(np.random.permutation(df.index))
9 print('Data shuffled successfully! 🍷') # طباعة
10
11 # --- الحفظ كملف CSV ---
12 csv_filename = 'movie_data.csv'
13 print(f'Saving data to {csv_filename}...') # طباعة
14 # index=False pandas حتى index مكان ما يحفظ الـ
15 df_shuffled.to_csv(csv_filename, index=False, encoding='utf-8')
16 print(f'Data saved successfully to {csv_filename}') # طباعة
17
18 # --- التأكد (اختياري لكن مفيد) ---
19 try:
20     print('\n--- Verifying Saved File ---') # طباعة
21     df_check = pd.read_csv(csv_filename, encoding='utf-8')
22     print('First 3 rows of the file:') # طباعة
23     print(df_check.head(3))
24     print(f'Total data shape: {df_check.shape}') # (50000, 2)
25     print('Data looks OK! ✅') # طباعة
26 except Exception as e:
27     # نطبع خطأ التأكد
28     print(f'Error during file verification: {e}')

```

Shuffling data...

Data shuffled successfully! 🍷

Saving data to movie_data.csv...

Data saved successfully to movie_data.csv

\n--- Verifying Saved File ---

First 3 rows of the file:

	review	sentiment
0	5 minutes into this movie I was hyperventilati...	0
1	I watched this again after having not seen it ...	1
2	What I liked best about this feature-length an...	1

Total data shape: (50000, 2)

Data looks OK! ✅

وبها الخطوات نكون جهزنا بيانات IMDb في ملف CSV واحد مرتب وجاهز! 😎 الحين صار عندنا الأساس اللي نبني عليه الخطوة الجاية كيف نتعامل مع النص نفسه؟ هنا يجي دور تقنيات زي حقيبة الكلمات (Bag-of-Words)...

طيب، بما إن بيانات IMDb صارت جاهزة عندنا في ملف مرتب، كيف نخلي الكمبيوتر يفهم الكلام اللي جوا الـ reviews؟
🤔 النصوص بحد ذاتها ما تدخل مباشرة للموديلات الرياضية. نحتاج طريقة نحول فيها الكلمات لأرقام أو خصائص (Features) أول وأبسط طريقة مشهورة هي نموذج حقيبة الكلمات أو الـ (Bag-of-Words (BoW) خلونا نتعرف عليه 📌

وش هو الـ (Bag-of-Words (BoW؟ 🤔 هو ببساطة طريقة سهلة ومشهورة عشان نستخلص خصائص (Features) من النصوص ونحولها لأرقام يقدر موديل تعلم الآلة يتعامل معاها. الفكرة الأساسية هي تمثيل كل نص كحقيبة فيها كلمات بدون الاهتمام بترتيبها، بس يهمننا وش الكلمات الموجودة وكم مرة تكررت.

كيف نسوي الـ (Bag-of-Words (BoW؟ 🤔 الخطوات الأساسية بسيطة ومتراصة. 1. نجمع كل الكلمات الفريدة من كل الـ reviews ونسوي معجم (Vocabulary) خاص فينا قائمة بكل الكلمات الممكنة. 2. لكل review ، ننشئ متجه (Vector) بنفس حجم (طول) الـ Vocabulary هذا. 3. الـ Vector هذا يعكس معلومات عن كلمات الـ review اللي من الـ Vocabulary بنتكلم عن كيف بالضبط في التريد الجاي مع الـ (TF) Term Frequency.

طيب، كيف نسوي هذا الكلام عملياً في بايثون بسهولة؟ 🤔 مكتبة scikit-learn توفر لنا أداة رهيبة اسمها CountVectorizer تسوي لنا كل شغل الـ Bag-of-Words هذا! خلونا نشوف كيف نستخدمها بمثال 📌. هذا مثال بسيط يوضح استخدام CountVectorizer:

```

1 import numpy as np
2 from sklearn.feature_extraction.text import CountVectorizer
3
4 # 1. instance من الـ Vectorizer
5 # CountVectorizer (stopword زي إزالة الـ) عنده خيارات كثيرة ممكن تعديلها بعدين
6 count = CountVectorizer()
7
8 # 2. نعرف بعض المستندات (الجملة) كمثال عشان نطبق عليها
9 docs = np.array([
10     'The sun is shining',
11     'The weather is sweet',
12     'The sun is shining, the weather is sweet, and one and one is two'
13 ])
14
15 # 3. تطبيق fit_transform:
16 # .fit() الـ Vocabulary يتعلم الـ docs
17 # .transform() الـ docs لي matrix على الـ Vocabulary بناءً
18 # fit_transform() بسوي الخطوتين بوا
19 bag = count.fit_transform(docs)
20
21 # bag الـ BoW (Sparse Matrix) الحين يحتوي على تمثيل الـ
22 print("Bag-of-Words model created successfully using CountVectorizer!")

```

Bag-of-Words model created successfully using CountVectorizer!

طيب وش الـ Vocabulary اللي استنتجه الـ CountVectorizer من الـ docs حققتنا؟ نقدر نشوفه كـ dictionary يربط كل كلمة بـ index رقمي خاص فيها باستخدام : vocabulary_ لاحظ في المخرجات : 💡 كيف كل كلمة فريدة (token) أخذت رقم (index) خاص فيها 🤖 .



```
1 # الـ vocabulary_ attribute نطبع الـ
2 print("Vocabulary created:")
3 print(count.vocabulary_)
```



```
1 Vocabulary created:
2 {'the': 6, 'sun': 4, 'is': 1, 'shining': 3, 'weather': 8, 'sweet': 5, 'and': 0, 'one': 2, 'two': 7}
```

والـ Feature Vectors كيف شكلها؟ الـ bag اللي رجع لنا هو مصفوفة قليلة الكثافة (Sparse Matrix) عشان توفر مساحة. ولو حبيينا نشوفها بشكل أوضح كأرقام مفهومة نقدر نحولها لـ numpy array باستخدام : toarray(). لاحظ في المخرجات : 💡 كل صف يمثل مستند، وكل عمود يمثل كلمة من الـ Vocabulary بنفس ترتيب الـ index اللي شفاها فوق)، والقيم هي عدد تكرار الكلمة في المستند 🙌✅ (Term Frequency).



```
1 الـ bag لـ numpy array ونطبعه كـ
2 print("\nFeature vectors (Bag-of-Words matrix)")
3 print(bag.toarray())
```

Feature vectors (Bag-of-Words matrix):

```
[[0 1 0 1 1 0 1 0 0]
 [0 1 0 0 0 1 1 0 1]
 [2 3 2 1 1 1 2 1 1]]
```

الزبدة : تعرفنا فيه على أساسيات تحليل المشاعر وأهميته، وجهزنا بيانات IMDb المهمة لمثالنا العملي، وبدأنا أول خطوة في تحويل النصوص لأرقام بنموذج ✓ Bag-of-Words وعشان ما نطول عليكم الشرح زيادة، بنكمل في الثريد القادم بإذن الله مع تحسينات مهمة زي الـ TF-IDF خليكم قريبين 🙏🔥 !

--

تحليل المشاعر مو بس عدّ كلمات... أحياناً كلمة وحدة تغيّر كل المعنى 😲 !ليش "رائع" أهم من "فيلم" حتى لو تكررت أقل؟ هنا يجي دور TF-IDF ، الذكاء اللي يخلي الخوارزميات تفهمك صح !الثريد هذا بيشرح لك الفكرة بالتفصيل، يفضل حفظه والرجوع له وقت تركيزك 🕒👉

TF-IDF

TERM FREQUENCY - INVERSE DOCUMENT FREQUENCY

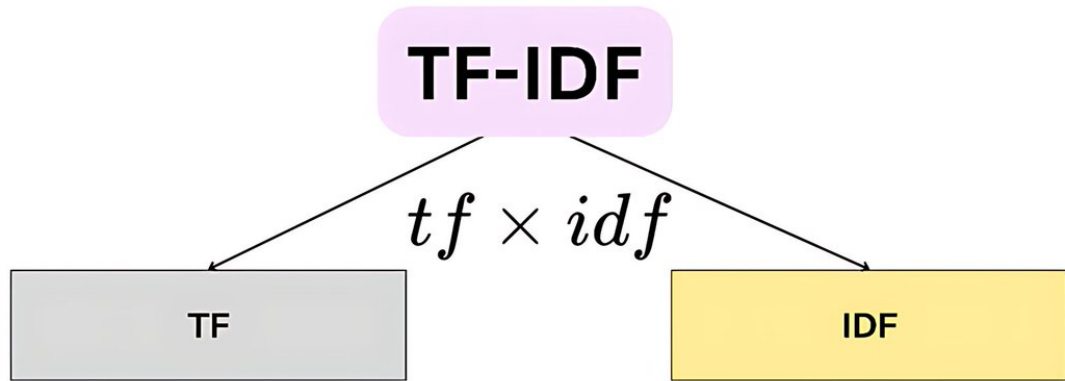


$$\text{TF-IDF} = \text{TF} \times \log \left(\frac{N}{df} \right)$$

N = number of documents **df** = number of documents that contain the term

توقفنا في الثريد اللي راح عند إننا حولنا النصوص لـ Feature Vectors باستخدام الـ Bag-of-Words ، وكنا وصلنا لإننا طلعنا مصفوفة. وبعدها حولناها لـ NumPy array عشان نشوفها بوضوح ⚡ مثلاً، المصفوفة توضح تكرار الكلمات في كل نص بطريقة منظمة، كأنها جدول يعرض كل كلمة وعددها في كل مستند، مما يساعدنا نلاحظ الأنماط بكل سهولة!

الحين خلونا نركز على كيف تطور المصفوفة ونخليها أقوى باستخدام الـ TF-IDF! هي حاصل ضرب الـ TF يعني كم مرة تكررت الكلمة في النص مع الـ IDF يعني ندرة الكلمة في مجموع النصوص. وهذه الطريقة بتعطي أهمية أكبر للكلمات اللي تميز كل نص عن الثاني. خلونا نفهم كل جزء وش يعني بالتفصيل، ونشوف كيف الـ TF-IDF تقدر تحسن تحليل النصوص.



$$tf(t,d) = count(t,d) \quad idf(t) = \log\left(\frac{1 + N_{documents}}{1 + df(t)}\right) + 1$$

وش يعني الـ (TF) Term Frequency؟ 🤔 الـ TF هو مقياس يحسب كم مرة تكررت كلمة معينة في نص واحد. يعني لو عندنا مجموعة نصوص ونبيغي نعرف أي نص أقرب لجملته مثل "Data Science is awesome" ، نقدر نعد كم مرة ظهرت كلمات مثل "Data" ، "Science" ، أو "awesome" في كل نص. كل ما تكررت الكلمة أكثر، كل ما زاد وزنها.

المعادلة:

$$tf(t,d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$$

▪ $f(t,d)$: عدد مرات ظهور الكلمة t في المستند d .

▪ $\sum f(t',d)$: مجموع تكرار كل الكلمات في المستند d .

مثال عملي: لو عندنا مستند فيها 100 كلمة، وكلمة "awesome" تكررت 5 مرات، ف $tf(awesome, d) = 5 / 100 = 0.05$ صحيح! الـ TF يحسب النسبة عشان بخلي الأوزان متناسقة مهما كان طول النص. هالشي يساعدنا نقارن بين نصوص بأطوال مختلفة بكل سهولة 👍 .

وش يعني ال-IDF (Inverse Document Frequency) ؟ 🤔 ال-IDF هو Inverse Document Frequency أو ال-IDF هو اللي يخلي الكلمات النادرة أهم من الكلمات الشائعة. الكلمات كيف "is" أو "the" تظهر في كل النصوص تقريبًا، فما تعطينا معلومات مميزة. ال-IDF يقلل وزن الكلمات الشائعة ويزود وزن الكلمات النادرة مثل "fantastic" أو "awful". المعادلة:

$$idf(t, d) = \log \frac{n_d}{1 + df(d, t)}$$

▪ n_d : هو إجمالي عدد المستندات

▪ $df(d, t)$: هو عدد المستندات d اللي تحتوي على الكلمة t

▪ $1 +$: عشان نتجنب القسمة على صفر لو الكلمة موجودة في أي نص.

▪ ال- $\log$: يساعد يقلل تأثير الفروقات الكبيرة .

مثال عملي :لو عندنا 50 ألف مستند ▪ $(n_d = 50,000)$ كلمة "movie" موجودة في 40 ألف مستند ($d=40,000$) ، ف :
 ▪ $idf(movie) = \log(50,000 / (40,000 + 1)) \approx 0.22$ كلمة "awesome" موجودة في 500 مستند، ف :
 $idf(awesome) = \log(50,000 / (500 + 1)) \approx 4.6$ ال-IDF تخلي الكلمات النادرة مثل "awesome" أهم بكثير من كلمة متكررة زي "movie"

كيف نجمع ال-TF-IDF ؟ 🤔 ال-IDF يدمج ال-TF وال-IDF عشان يعطي كل كلمة وزن يعكس أهميتها في نص معين وندرتها في المجموعة الكلية. المعادلة : $tf(t, d)$: تكرار الكلمة في النص (من ال- $idf(t, d)$) . ▪ (TF) وزن الكلمة بناءً على ندرتها (من ال-IDF).

$$tf-idf(t, d) = tf(t, d) \times idf(t, d)$$

لتوضيح الفكرة أكثر خلونا نشوف مثال كامل من أول ما نطلع ال feature vectors إلين ما نطبق ال-TF-IDF. نطبق ال-TF-IDF :
 ال-IDF لأربع مستندات خطوة بخطوة! المستندات هي : "The sun is bright today." ▪ "d1" : "The sky is blue." ▪ :

d2" ■ The sun in the sky is bright." : d3" ■ We can see the shining sun, the bright sun." :
d4"

الخطوة الأولى: نحذف الـ Stopwords كيف "the" ، "is" ، "in" بعد ما نحذفها، المستندات بتصير "d1: sky blue." :
■ sun bright today." : d2" ■ sun sky bright." : d3" ■ can see shining sun, bright sun." : d4"
الكلمات المهمة هي. blue, bright, can, see, shining, sky, sun, today.

$$f_{t,d}$$

	blue	bright	can	see	shining	sky	sun	today
1	1	0	0	0	0	1	0	0
2	0	1	0	0	0	0	1	1
3	0	1	0	0	0	1	1	0
4	0	1	1	1	1	0	2	0
n_t	1	3	1	1	1	2	3	1

الخطوة الثانية: نحسب الـ (Term Frequency (TF نسوي مصفوفة توضح تكرار كل كلمة في كل مستند، وبعدها نسوي Normalize للصفوف عشان مجموعها يصير 1. مثلاً: في d1: sky blue ، كلمة "sky" تكررت مرة و "blue" تكررت مرة، فالـ TF لكل وحدة = $2/1 = 0.5$. نحسب لكل المستندات بنفس الطريقة 😊 !

$f_{t,d}$

	blue	bright	can	see	shining	sky	sun	today
1	1	0	0	0	0	1	0	0
2	0	1	0	0	0	0	1	1
3	0	1	0	0	0	1	1	0
4	0	1	1	1	1	0	2	0

→

$tf(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$

	blue	bright	can	see	shining	sky	sun	today
1	1/2	0	0	0	0	1/2	0	0
2	0	1/3	0	0	0	0	1/3	1/3
3	0	1/3	0	0	0	1/3	1/3	0
4	0	1/6	1/6	1/6	1/6	0	1/3	0

الخطوة الثالثة 🤖: نحسب الـ IDF (Inverse Document Frequency) بنشوف كل كلمة ظهرت في كم مستند، وبعدها نستخدم الصيغة:

$f_{t,d}$

	blue	bright	can	see	shining	sky	sun	today
1	1	0	0	0	0	1	0	0
2	0	1	0	0	0	0	1	1
3	0	1	0	0	0	1	1	0
4	0	1	1	1	1	0	2	0
n_t	1	3	1	1	1	2	3	1

→

$idf(t, D) = \log_{10} \frac{N}{n_t}$

	blue	bright	can	see	shining	sky	sun	today
	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

$\log_{10} \frac{4}{1} = 0.602$

$\log_{10} \frac{4}{3} = 0.125$

الخطوة الأخيرة 😎: نحسب الـ TF-IDF! نضرب قيم الـ TF في الـ IDF لكل كلمة في كل مستند. مثلاً: لو الـ TF لكلمة "sun" في d2 = 0.33، والـ IDF لـ "sun" = 0، فـ $tf-idf(sun, d2) = 0.33 * 0 = 0$. وبكذا نعرف وزن كل كلمة في كل مستند، والكلمات النادرة تاخذ وزن أعلى! ✓

$tf(t, d)$								
	blue	bright	can	see	shining	sky	sun	today
1	1/2	0	0	0	0	1/2	0	0
2	0	1/3	0	0	0	0	1/3	1/3
3	0	1/3	0	0	0	1/3	1/3	0
4	0	1/6	1/6	1/6	1/6	0	1/3	0

×

$idf(t, D)$								
	blue	bright	can	see	shining	sky	sun	today
	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

$$tfidf(t, d, D) = tf(t, d) \cdot idf(t, D)$$

→

$tfidf(t, d, D)$								
	blue	bright	can	see	shining	sky	sun	today
1	0.301	0	0	0	0	0.151	0	0
2	0	0.0417	0	0	0	0	0.0417	0.201
3	0	0.0417	0	0	0	0.100	0.0417	0
4	0	0.0209	0.100	0.100	0.100	0	0.0417	0

- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
- Most important word for each document is highlighted

و ش هي الـ Normalization في الـ TF-IDF ؟ 🤔 الـ TF-IDF ينتج أوزان مختلفة للكلمات، بس لو قارنا بين نصوص بأطوال مختلفة، الأوزان ممكن تكون غير عادلة. الـ Normalization تحديدًا الـ L2-normalization بتخلي طول كل متجه يساوي 1، عشان نقدر نقارن بين النصوص بسهولة... تمامًا زي ما الميزان يضبط التوازن بين الأوزان!

ليه الـ TF-IDF مهم؟ 🤔 الـ TF-IDF يحل مشكلة الـ Bag-of-Words اللي كان يعطي وزن كبير للكلمات الشائعة كيف "fantastic" أو "movie" اللي ما تعبر عن مشاعر. الـ TF-IDF يزود وزن الكلمات النادرة والمعبرة كيف "disappointing". الـ TF-IDF يقلل وزن الكلمات الشائعة اللي في كل النصوص. يساعد موديلات تعلم الآلة تصنف المراجعات (إيجابي/سلبي) بدقة أعلى. مثال: لو عندنا مستند "The movie was fantastic"، الـ TF-IDF بيعطي "fantastic" وزن كبير لأنها نادرة وتعبر عن إحساس قوي 🔥.

خلونا نشوف كيف نطبق هذا كله ببايثون؟ بنحسب الـ TF-IDF باستخدام TfidfTransformer من scikit-learn:

```

1 # TF-IDF: عشان نحسب الـ TfidfTransformer باعدادنا بتستخدم مكتبة
2 from sklearn.feature_extraction.text import TfidfTransformer
3
4 # الـ L2-normalization، عشان نحسب الـ IDF، norm='l2'، smooth_idf=True، use_idf=True، بتستخدم الـ TF-IDF، بتعمل أداة الـ
5 tfidf = TfidfTransformer(use_idf=True, norm='l2', smooth_idf=True)
6
7 # بتعطى عدد الترميز في النماذج، عشان تكون النتيجة واضحة numpy، بتستخدم مكتبة
8 import numpy as np
9 np.set_printoptions(precision=2)
10
11 # عشان نحسب الـ TF-IDF، بتحولها الـ CountVectorizer، بتعمل تكرارات بين الـ TF-IDF، الـ count، الـ
12 print(tfidf.fit_transform(count.fit_transform(docs)).toarray())

```

```
[[ 0.    0.43  0.    0.56  0.56  0.    0.43  0.    0. ]  
 [ 0.    0.43  0.    0.    0.    0.56  0.43  0.    0.56]  
 [ 0.5   0.45  0.5   0.19  0.19  0.19  0.3   0.25  0.19]]
```

الزبدة 🔥: فهمنا بالتفصيل أسرار معادلات TF ، ال-IDF ، والTF-IDF ، وكيف ال-Normalization تسوي توازن ذكي عشان تضمن بيانات دقيقة وموثوقة! الخطوة الجاية؟ بنبدأ في تنظيف النصوص بعناية واستخدام ال-N-grams عشان نرفع دقة التحليل! احفظوا الثريد، وخلّوكم جاهزين 🙌😎!